1) Цель

Освоить работу с pandas для загрузки и обработки данных.

Освоить работу с библиотеками визуализации данных (matplotlib/seaborn).

Освоить базовые навыки работы с данными для машинного обучения.

2) Задание

Работу необходимо выполнить в jupyter-блокноте exercises.ipynb. Рекомендуется выполнять задания по порядку.

3) Результат выполнения

В результате были выполнены все обязательные задания и одно бонусное задание на 1 балл. Задание прошло автоматическое тестирование на 8.0 / 10.0

4) Исходный код

exercises.ipynb

```
# # Задание 1. Инициализация среды. Загрузка и осмотр данных


# ## Шаг 0. Настройка среды выполнения
# Если вы выполняете задание на локальной машине, перед выполнением задания создайте виртуальную среду, активируйте её и установите библиотеки `pandas`, `numpy`, `matplotlib` и `seaborn`.
#
# Если вы выполняете задание в Colab или Kaggle, активируйте среду выполнения.


# ## Шаг 0.1. Импорт библиотек
# Выполните ячейку ниже, чтобы импортировать все необходимые для работы библиотеки.


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# ## Шаг 1. Определение номера варианта и скачивание датасета
```

```
# Перейдите в Google Таблицу со списком студентов. Найдите своё ФИО и запомните порядковый номер (поле № п/п) в первом столбце. Заполните его в ячейке ниже и выполните ячейку. Если вы не можете найти себя в списке — обратитесь к преподавателю.
#


Student_ID = 7

# Теперь выполните следующую ячейку. Она вычислит номер варианта и выведет ссылку на датасет.


datasets = [
    ('US Air Carrier Market 2019',
     'https://raw.githubusercontent.com/markpolyak/datasets/refs/heads/main/data/
     aircarrier_market_us_2019.zip'),
    ('Video Game Sales with Ratings',
     'https://www.kaggle.com/datasets/rush4ratio/video-game-sales-with-ratings'),
    ('Titanic',
     'https://www.kaggle.com/datasets/yasserh/titanic-dataset'),
    ('Adult Income (Census Income)',
     'https://www.kaggle.com/datasets/uciml/adult-census-income'),
    ('Spotify Tracks Dataset',
     'https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset'),
    ('Melbourne Housing Market',
     'https://www.kaggle.com/datasets/anthonypino/melbourne-housing-market'),
    ('NYC Airbnb Listings',
     'https://www.kaggle.com/datasets/dgomonov/new-york-city-airbnb-open-data'),
    ('Airline Passenger Satisfaction',
     'https://www.kaggle.com/datasets/teejmahal20/airline-passenger-satisfaction'),
]

dataset_id = None if Student_ID is None else (Student_ID - 1) % len(datasets)
if dataset_id is None:
    print("ОШИБКА! Не указан Student_ID")
else:
    print(f"Ваш датасет № {dataset_id + 1}: {datasets[dataset_id][0]}")
    print(f"Ссылка: {datasets[dataset_id][1]}")
    print(f"Подробности в файле datasets_info.md")
```

Скачайте датасет. Если вам дана ссылка на файл (`.csv`, `.zip`, `.tsv` и т.д.), скачайте его с помощью команды `!wget <dataset_url>` или `curl ...`. При необходимости разархивируйте датасет командами `!unzip`, `!tar` и др.
#
Если вам дана ссылка на kaggle.com, перейдите по ней и скачайте датасет кнопкой Download. В раскрывающемся списке «Download via» выберите cUrl и скопируйте готовую команду в ячейку ниже.
#
Примечание: в Jupyter-ноутбуке можно использовать команды bash, поставив в начале ячейки `!`.

BEGIN YOUR CODE

!curl -L -o new-york-city-airbnb-open-data.zip https://www.kaggle.com/api/v1/datasets/download/dgomonov/new-york-city-airbnb-open-data

!unzip new-york-city-airbnb-open-data.zip
!tar ...
!gunzip ...

END YOUR CODE

Шаг 2. Загрузка данных
#

Загрузите датасет в `pandas.DataFrame`, сохраните его в переменной `df`. Проинспектируйте датасет, используя `df.head()`.

BEGIN YOUR CODE
df = pd.read_csv("AB_NYC_2019.csv")

df.head()
END YOUR CODE

Сконвертируйте названия столбцов в нижний регистр, после этого создайте переменную `columns` и сохраните в неё список столбцов.

BEGIN YOUR CODE

columns = df.columns.str.lower().to_list()
END YOUR CODE

print(columns)

Выведите информацию о датафрейме с помощью функции `.info()`. Обратите внимание на типы столбцов и количество непустых значений в каждом из них.

BEGIN YOUR CODE

df.info()

END YOUR CODE

Шаг 3. Анализ датафрейма
#

Проверьте, есть ли в данном датасете пропущенные значения? Удалите из датафрейма **строки** с пропущенными значениями.
#
ВАЖНО: не перезаписывайте исходную переменную `df`, вместо этого сохраните новый датафрейм в переменную `df_dropna`.

BEGIN YOUR CODE

df_dropna = df.dropna()
END YOUR CODE

Сгенерируйте сводную статистику по вашему датасету с помощью встроенной функции `.describe()`.

BEGIN YOUR CODE

df_dropna.describe(include="all")
END YOUR CODE

Вопрос для размышления. Ответьте на следующие вопросы, записав ответы в переменные ниже:
#
- `reflection_1_missing` — какова доля пропущенных значений в датасете (в % от общего числа ячеек)? Считаете ли вы эту долю значительной? Почему?
- `reflection_1_mean_median` — для каких числовых признаков среднее и медиана сильно расходятся? Что это говорит о форме распределения?

BEGIN YOUR CODE

print((df.isnull().sum().sum() / df.size) * 100)

reflection_1_missing = """Процент пропусков около 2.6%, что незначительно в масштабах датасета, однако пропуски сконцетрированы в last_review и reviews_per_month,
что может иметь значение при анализе данных на основе этих признаков""" # str: ваш ответ текстом
reflection_1_mean_median = """ Для признаков price minimum_nights number_of_reviews calculated_host_listings_count availability_365.
Эта информация говорит о выбросах в данных, которые и завышают среднее, например, слишком высокая цена жилья""" # str: ваш ответ текстом
END YOUR CODE

Дополнительное задание *. Заполните пропуски в датасете
#
ВАЖНО: не является обязательным, но даёт дополнительные баллы.
#
Для заполнения пропусков используйте моду для категориальных признаков и медиану для числовых. Сохраните результат в переменную `df_fillna`.

BEGIN YOUR CODE
df_fillna = df.copy()

num_cols = df_fillna.select_dtypes(include=['number']).columns
df_fillna[num_cols] = df_fillna[num_cols].fillna(df_fillna[num_cols].median())

cat_cols = df_fillna.select_dtypes(include=['object']).columns
df_fillna[cat_cols] = df_fillna[cat_cols].fillna(df_fillna[cat_cols].mode().iloc[0])

END YOUR CODE

Задание 2. Работа с числовыми признаками

Шаг 1. Отбор числовых признаков

Определите числовые признаки датасета. Запишите их названия в список `numeric_features`. После этого отделите от датафрейма `df_dropna` часть со всеми числовыми признаками.
#
Примечание: ориентируйтесь на смысл признака, а не только на тип в pandas — например, год выпуска может храниться как `int`, но по смыслу быть категориальным. В таких случаях принимайте решение самостоятельно.
#
ВАЖНО: не перезаписывайте `df_dropna`, запишите результат в переменную `df_numeric`.
#

BEGIN YOUR CODE
```
numeric_features = [
    "price",
    "minimum_nights",
    "number_of_reviews",
    'reviews_per_month',
    "availability_365",
    "calculated_host_listings_count",
    "latitude",
    "longitude"
]

df_numeric = df_dropna[numeric_features].copy()
```
END YOUR CODE

df_numeric

Шаг 2. Визуализация распределений

```
# Для каждого числового признака визуализируйте его распределение, используя гистограммы. Поместите название признака в заголовок. Сохраните объект Figure в переменную `fig_numeric_distributions`.
#
# Подсказка: используйте `plt.subplots` для размещения нескольких графиков на одной фигуре.


### BEGIN YOUR CODE
fig_numeric_distributions, axes = plt.subplots(nrows=4, ncols=2, figsize=(10, 10))
# задайте нужное число строк и столбцов

axes_to_flat = axes.flatten()

for i, col in enumerate(numeric_features):
    axes_to_flat[i].hist(df_numeric[col], bins=20)

    axes_to_flat[i].set_title(col)
    axes_to_flat[i].set_xlabel('Значение')
    axes_to_flat[i].set_ylabel('Частота')

plt.tight_layout()
plt.show()

### END YOUR CODE

# **Вопрос для размышления.** Запишите ответ в переменную `reflection_2_distributions`:
#
# Опишите форму распределения для каждого числового признака: симметричное или скошенное, унимодальное или мультимодальное? Есть ли очевидные выбросы, заметные на гистограмме?


### BEGIN YOUR CODE
reflection_2_distributions = """Все 6 первых распределений являются скошенными унимодальными по виду, на графиках видно монолитный столбец с правым хвостом, что означает
о скоплении выбросов справа. Широта и долгота больше напоминают нормальное распределение, так как вся недвижимость находится в Нью Йорке."""  # str: ваш ответ текстом
```

```
### END YOUR CODE

# ## Шаг 3. Поиск корреляции между числовыми признаками


# Вычислите корреляции по Пирсону и по Спирмену между всеми парами числовых признаков. Сохраните матрицы корреляций в переменные `corr_pearson` и `corr_spearman` соответственно.
#
# Подсказка: используйте метод `df_numeric.corr(method=...)`. Документация: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.corr.html


### BEGIN YOUR CODE
corr_pearson = df_numeric.corr(method='pearson')
corr_spearman = df_numeric.corr(method='spearman')
### END YOUR CODE

# Для корреляций по Пирсону постройте тепловую карту от -1 до 1, с центром в 0. Укажите названия признаков для строк и столбцов. Сохраните Figure в переменную `fig_heatmap_pearson`.
#
# Подсказка: используйте `seaborn.heatmap()`.


### BEGIN YOUR CODE
fig_heatmap_pearson, ax = plt.subplots()

sns.heatmap(
    corr_pearson,
    annot=True,
    cmap='coolwarm',
    vmin=-1,
    vmax=1,
    center=0,
    ax=ax
)

### END YOUR CODE
```

```
# Аналогично для корреляций по Спирмену. Сохраните Figure в переменную `fig_heatmap_spearman`.


### BEGIN YOUR CODE
fig_heatmap_spearman, ax = plt.subplots()

sns.heatmap(
    corr_spearman,
    annot=True,
    cmap='coolwarm',
    vmin=-1,
    vmax=1,
    center=0,
    ax=ax
)

### END YOUR CODE

# **Вопрос для размышления.** Запишите ответы в переменные ниже:
#
# - `reflection_3_pearson` — какая пара признаков имеет наибольшую корреляцию по Пирсону? Как вы интерпретируете эту связь содержательно?
# - `reflection_3_difference` — есть ли пары признаков, у которых корреляция по Пирсону и по Спирмену заметно отличается? Как вы думаете, с чем это связано?


### BEGIN YOUR CODE
reflection_3_pearson = """Наиболее сильную связь по Пирсону можно увидеть у number_of_reviews и reviews_per_month, что и логично,
так как если растёт количество отзывов в месяц, то и общее количество растёт. Отсюда и видно корреляцию по Пирсону."""  # str: ваш ответ текстом
reflection_3_difference = """Можно обратиться к тем же признакам, что и ранее. Отличие Пирсона и Спирмена составляет около 0.2 .Это связано в первую очередь с тем,
что Спирмен ищеет нелинейную корреляцию в отличие от Пирсона, что и прослеживает в такой паре признаков."""  # str: ваш ответ текстом
### END YOUR CODE

# ## Шаг 4. Вычисление статистик
```

Для каждого числового признака вычислите среднее, медиану и стандартное отклонение.
#
Запишите результаты в словари `means`, `medians` и `stds`, где ключ — название признака (как в датафрейме), значение — соответствующая статистика.

BEGIN YOUR CODE
means = {}
medians = {}
stds = {}

means = df_numeric.mean().to_dict()
medians = df_numeric.median().to_dict()
stds = df_numeric.std().to_dict()

END YOUR CODE

Шаг 5. Стандартизация признаков

Выполните стандартизацию для каждого числового признака:
#
$$
z = \dfrac{x - \mu}{\sigma}
$$
#
где μ — среднее по признаку, σ — стандартное отклонение.
#
Подсказка: используйте статистики из предыдущего шага.
#
Сохраните результат в переменную `df_standart`.

BEGIN YOUR CODE
df_standart = (df_numeric - pd.Series(means)) / pd.Series(stds)
END YOUR CODE

Повторите шаг 2 для полученного датафрейма — постройте гистограммы распределений. Сохраните Figure в переменную `fig_standart_distributions`.

```
### BEGIN YOUR CODE
fig_standart_distributions, axes = plt.subplots(nrows=4, ncols=2, figsize=(10, 10))

axes_to_flat_standart = axes.flatten()

for i, col in enumerate(numeric_features):
    axes_to_flat_standart[i].hist(df_standart[col], bins=20)

    axes_to_flat_standart[i].set_title(col)
    axes_to_flat_standart[i].set_xlabel('Значение')
    axes_to_flat_standart[i].set_ylabel('Частота')

plt.tight_layout()
### END YOUR CODE
```

Вопрос для размышления. Запишите ответ в переменную `reflection_5_standardization`:
#
Изменилась ли форма распределений после стандартизации? Изменились ли среднее и стандартное отклонение? Почему?

```
### BEGIN YOUR CODE
reflection_5_standardization = """Форма распределения всех признаков осталась та же. Среднее и стандартное отклоление изменились так как данная стандартизация
приводит распределение к среднему 0 и отколнением 1."""  # str: ваш ответ текстом
### END YOUR CODE
```

Задание 3. Работа с категориальными признаками

Шаг 1. Отбор категориальных признаков

Определите категориальные признаки датасета. Запишите их названия в список `cat_features`. После этого отделите от датафрейма `df_dropna` часть со всеми категориальными признаками.
#
ВАЖНО: не перезаписывайте `df_dropna`, запишите результат в переменную `df_cat`.

BEGIN YOUR CODE
drop_cats = ["id", "host_id"] # Не относятся ни к одному типу признака

cat_features = [col for col in df_dropna.columns if col not in numeric_features and col not in drop_cats]

df_cat = df_dropna[cat_features].copy().astype(str)

END YOUR CODE

Шаг 2. Визуализация распределений категориальных признаков

Для каждого категориального признака визуализируйте частоту встречаемости каждой категории в виде столбчатой диаграммы. Подпишите оси и укажите название признака в заголовке. Сохраните Figure в переменную `fig_cat_distributions`.
#
Подсказка: используйте `value_counts()` для подсчёта частот и `sns.barplot` или `.plot(kind='bar')` для визуализации.
#
Примечание: если у признака более 20 категорий — отображайте только топ-20.

BEGIN YOUR CODE
nrows = (len(cat_features) + 1) // 2

fig_cat_distributions, axes = plt.subplots(nrows=nrows, ncols=2, figsize=(15, 15))
axes_to_flat_cat = axes.flatten()

for i, col in enumerate(cat_features):
    counts = df_cat[col].value_counts().head(20)

```
    sns.barplot(x=counts.values, y=counts.index, ax=axes_to_flat_cat[i], orient='h')

    axes_to_flat_cat[i].set_title(col)
    axes_to_flat_cat[i].set_xlabel('Количество')
    axes_to_flat_cat[i].set_ylabel('Категория')

plt.tight_layout()
plt.show()

### END YOUR CODE

# **Вопрос для размышления.** Запишите ответ в переменную
`reflection_3_cat_imbalance`:
#
# Есть ли в ваших категориальных признаках сильный дисбаланс (одна
категория встречается значительно чаще других)? Для каких признаков это
наблюдается и насколько выражен дисбаланс?



### BEGIN YOUR CODE
reflection_3_cat_imbalance = """Дисбаланс выражен для многих признаков,
такие как room_type, neighbourhood, neighbourhood_group. В данных признаках
есть категории количество которых
больше других в десятки раз (Entire home & private room) (Manhattan &
Broklyn)"""  # str: ваш ответ текстом
### END YOUR CODE

# ## Шаг 3. One-hot encoding



# Примените one-hot кодирование для ваших категориальных признаков.
Подсказка: используйте `pandas.get_dummies(...)`.
#
# **ВАЖНО**: не перезаписывайте `df_cat`, запишите результат в
переменную `df_ohe`.



### BEGIN YOUR CODE

df_ohe = pd.get_dummies(df_cat)
```

df_ohe.head()
END YOUR CODE

Вопрос для размышления. Запишите ответ в переменную
`reflection_3_ohe`:
#
Как изменилось число столбцов в датафрейме после one-hot кодирования?
Чем это потенциально опасно при обучении модели?

BEGIN YOUR CODE
reflection_3_ohe = """Количество столбцов в датафрейме увеличилось до
неимоверных числа. Это обусловленно такими признаками как name, last_review
и подобным
признакам с большим количеством уникальных признаков. Из-за такого
большого количества новых признаков модели будет сложно обнаружить
закономерности между ними, а
так же увеличится время обучения самой модели.""" # str: ваш ответ текстом
END YOUR CODE

Задание 4. Поиск и удаление выбросов

В этом задании используем ранее полученные датафреймы `df_standart` и
`df_ohe`.

Очистите данные от выбросов методом 3σ, применив его к
`df_standart`. Поскольку числовые данные уже стандартизированы, выбросом
считается **объект** (строка датафрейма), у которого хотя бы один признак
выходит за пределы интервала $(-3,\, 3)$, то есть $|z_{ij}| > 3$ хотя бы для
одного признака j.
#
Алгоритм:
1. Найдите маску строк-выбросов в `df_standart`.
2. Удалите их из `df_standart`, сохраните результат в `df_standart_f`.
3. Используя ту же маску (по индексу), удалите соответствующие строки из
`df_ohe`, сохраните в `df_ohe_f`.
4. Объедините `df_standart_f` и `df_ohe_f` по столбцам (`axis=1`),
сохраните в `df_filtered`.

```
#
# **ВАЖНО**: не перезаписывайте переменные `df_standart` и `df_ohe`.


### BEGIN YOUR CODE
# 1. Маска выбросов

df_mask = (df_standart.abs() > 3).any(axis=1)

# 2. Фильтрация числовых признаков
df_standart_f = df_standart[~df_mask]

# 3. Фильтрация категориальных признаков
df_ohe_f = df_ohe[~df_mask]

# 4. Объединение
df_filtered = pd.concat([df_standart_f, df_ohe_f], axis=1)

print(f"{len(df_standart)} - {df_mask.sum()} = {len(df_filtered)}")
### END YOUR CODE

# **Вопрос для размышления.** Запишите ответ в переменную
`reflection_4_outliers`:
#
# Сколько объектов было удалено как выбросы? Какой это процент от
исходного датасета? Как вы считаете, стоит ли их удалять, или лучше было бы
оставить — обоснуйте.


### BEGIN YOUR CODE
reflection_4_outliers = """Было удалено 2523 объектов, что примерно 6,5% от
исходного датасета. Удаление такой доли данных сделало данные
более стабильными для обучения модели. Выбросы бы только 'отвлекали' модель
от реальной закономерности"""  # str: ваш ответ текстом
### END YOUR CODE

# ### Дополнительное задание **. Поиск выбросов через расстояния до
центроидов кластеров
#
# **ВАЖНО**: не является обязательным, но даёт дополнительные баллы.
#
```

1. Выполните кластеризацию `df_standart` методом K-Means (`sklearn.cluster.KMeans`). Число кластеров выберите самостоятельно.
2. Для каждого объекта вычислите евклидово расстояние до центроида его кластера.
3. Внутри каждого кластера постройте распределение расстояний.
4. Отсеките выбросы по перцентилю (например, объекты дальше 95-го перцентиля). Обоснуйте выбор порога.
#
Сохраните итоговый датафрейм в переменную `df_filtered_kmeans`.

BEGIN YOUR CODE

df_filtered_kmeans = None

END YOUR CODE

5) Выводы

В процессе выполнения лабораторной работы были получены практические навыки по анализу данных с использованием языка Python и библиотек Pandas NumPy MatPlotLib SeaBorn